

Docker

- Installare docker
- **In Linux (Ubuntu / Debian / Mint)**
 - `sudo apt update`
 - `sudo apt install docker.io`
 - `sudo systemctl enable --now docker`
 - `sudo usermod -aG docker $USER` # opzionale: usare docker senza sudo
- **In MacOS**
 - Scaricare Docker Desktop: <https://www.docker.com/products/docker-desktop/>
 - Installare come una normale applicazione
- **In Windows:**
 - Scaricare Docker Desktop: <https://www.docker.com/products/docker-desktop/>
 - Richiede:
 - WSL2 abilitato
 - Virtualization attiva nel BIOS

Docker

- **Docker Command Line Interface (CLI)**
- **Verificare installazione**
 - `docker --version`
 - `docker run hello-world`
- **Comandi di base**
 - `docker pull <immagine>`
 - Es. `docker pull ubuntu`
 - Filesystem base, un layer RO base per creare container
 - Ogni layer è un tarball estratto

Docker

■ Comandi di base

- `docker pull <immagine>`
- `docker images #listare immagini`
- `docker rmi <image-id> #rimuovere immagini`

Docker

■ Comandi di base

- `docker pull <immagine>`
- `docker images` #listare immagini
- `docker rmi <image-id>` #rimuovere immagini
- `docker run -it ubuntu bash` #lanciare bash interattivamente
 - Crea un container dall'immagine ubuntu
 - Cerca un'immagine chiamata ubuntu in locale
 - Monta i layer dell'immagine (lowerdir)
 - Crea upperdir per la scrittura
 - Unisce tutto in OverlayFS (merged)
 - Crea namespace (PID, NET, MOUNT, UTS...)
 - Avvia bash come PID 1
 - `exit` per uscire

Docker

■ Comandi di base

- `docker pull <immagine>`
- `docker images` #listare immagini
- `docker rmi <image-id>` #rimuovere immagini
- `docker run -it ubuntu bash` #lanciare bash interattivamente
- `docker run --rm -it ubuntu bash` #per eliminarli all'uscita
- `docker exec [opzioni] <container> <comando>`
 - Permette di interagire con container già in esecuzione
 - Opzioni tipiche:
 - `-it` interattivo + terminale
 - `--user` esegue il comando come utente specifico
 - Esempi:
 - `docker exec -it mycontainer bash`
 - `docker exec mycontainer ps aux`

Docker

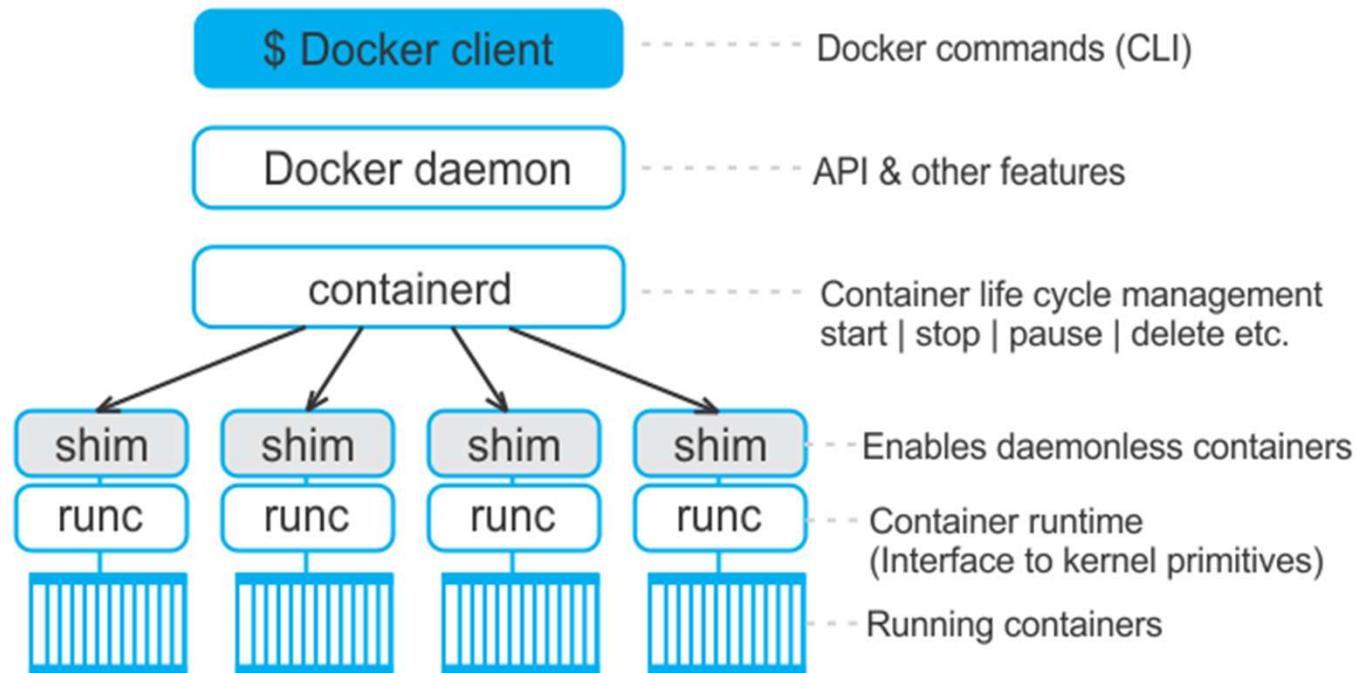
■ Comandi di base

- *docker pull <immagine>*
- *docker images #listare immagini*
- *docker rmi <image-id> #rimuovere immagini*
- *docker run -it ubuntu bash #lanciare bash interattivamente*
- *docker run --rm -it ubuntu bash #per eliminarli all'uscita*
- *docker rm <container-id> #rimuovere un container*
- *docker exec*
- *docker ps #mostrare container attivi*
- *docker ps -a #vede anche gli exited*
- *docker stop <container-id> #fermare un container*
- *docker start <container-id> #avviare un container*
- *docker container prune #rimuove tutti quelli spenti*

Docker

■ Docker engine

- Docker daemon interagisce con il docker client e ed esegue le azioni richieste
- Containerd è il supervisore dei container, runtime di alto livello che gestisce le operazioni sui container (start, stop, pause, delete etc.)
- Runc è il container runtime di basso-livello e l'interfaccia al kernel
- Shim (rondella) disaccoppia runc dal containerd



Docker

■ Docker engine

- Esempio
- `docker run -d ubuntu sleep 1000`
- `ps -ef | grep -E "dockerd | containerd | shim | sleep"`
- Però su WSL2 non funziona perché il kernel è su docker-desktop!

Docker

■ Immagini

- Listare le immagini disponibili
- `docker image ls`

■ Docker inspect

- Si possono vedere i dettagli di un'immagine:
- `docker inspect ubuntu`
 - Mostra i metadati associate ad un'immagine in formato json
 - ID dell'immagine, layer, comando di default (Cmd), variabili d'ambiente, working directory, user configurator, entrypoint, etc.

Docker

■ Immagini

- Listare le immagini disponibili
- `docker image ls`

■ Docker pull

- Per default Immagine scaricata da **Docker Hub**
- Quando si esegue: `docker pull ubuntu`
- Interpretato come: `docker pull docker.io/library/ubuntu`
- Esempio: `docker pull python:3.11-slim`
- `docker inspect python:3.11-slim`
- Dettagli con: `docker history python:3.11-slim`
- Si può indicare un registry (deposito) diverso
- Esempio: `docker pull quay.io/bitnami/python`

Docker

■ Immagini

- Listare le immagini disponibili
- `docker image ls`

■ Registri

- Registri contengono uno o più repository
- Repository contengono una o più immagini
- Repository ufficiali su Docker Hub

Docker

- Comandi di rete Docker essenziali
 - `docker network ls` #lista i network esistenti
 - Bridge di default, host, none
 - `docker network create` miarete
 - Crea una rete bridge virtuale
 - ai container collegati assegna IP privati (tipo 172.18.x.x)
 - i container su miarete si vedono tra loro come se fossero host diversi sulla stessa LAN

Docker

- Comandi di rete Docker essenziali
 - `docker network ls` #lista i network esistenti
 - Bridge di default, host, none
 - `docker network create miarete`
 - Crea una rete bridge virtuale
 - ai container collegati assegna IP privati (tipo 172.18.x.x)
 - i container su miarete si vedono tra loro come se fossero host diversi sulla stessa LAN
 - Collegare un container a una rete
 - `docker run --network miarete --name server ...`
 - `docker run --network miarete --name client ...`
 - `--network miarete` → mette il container nel net namespace collegato a quella rete
 - `--name server` → il nome diventa **hostname** e **DNS interno** sulla rete Docker (quindi client può contattare server per nome)
 - Ispezionare una rete
 - `docker network inspect miarete`

Docker

- Semplice rete in docker
 - Creazione namespace rete (bridge)
 - Server che ascolta su porta 5000
 - Client che si collega su porta 5000

- Preparazione rete
 - `docker pull ubuntu`
 - `docker network create miarete`
 - `docker network ls`

- **Lancio del container del server sulla rete create**
 - `docker run -it --name server --network miarete ubuntu bash`
 - `apt update`
 - `apt install -y netcat`
 - `nc -l -p 5000`

Docker

- Semplice rete in docker
 - Creazione namespace rete (bridge)
 - Server che ascolta su porta 5000
 - Client che si collega su porta 5000
- Preparazione rete
 - `docker pull ubuntu`
 - `docker network create miarete`
 - `docker network ls`
- **Lancio del container del client sulla rete create**
 - `docker run -it --name client --network miarete ubuntu bash`
 - `apt update`
 - `apt install -y netcat`
 - `nc server 5000`

Docker

- Semplice rete in docker
 - Creazione namespace rete (bridge)
 - Server che ascolta su porta 5000
 - Client che si collega su porta 5000

- Preparazione rete
 - `docker pull ubuntu`
 - `docker network create miarete`
 - `Docker network ls`

- **Chiusura container del client/server e rimozione della rete**
 - `docker stop server client`
 - `docker rm server client`
 - `docker network rm miarete`

Docker

Provare con codice c

- Scaricare server.c e client.c su teams
- Compilare i due file: `gcc server-bin.c -o sever-bin`
- Creare la rete: `docker network create miarete`
- Lanciare il container del server
 - `docker run --rm -it --name server-cont --network miarete ubuntu bash`
- In un nuovo terminale lanciare il container del client:
 - `docker run --rm -it --name client-cont --network miarete ubuntu bash`
- In un nuovo terminale copiare i file del client e del server:
 - `docker cp server-bin server-cont:/server-bin`
 - `docker cp client-bin client-cont:/app/client-bin`
- Nei rispettivi container eseguire
 - `./server-bin` e `./client-bin server-cont`
- Rimuovere la rete: `docker network rm miarete`

Docker

- Quando si usa una **rete Docker personalizzata** (come miarete), Docker crea **automaticamente un DNS interno** per quella rete
- Il DNS:
 - assegna un **hostname** a ogni container
 - l'hostname è esattamente il **nome del container**
 - risolve i nomi in **indirizzi IP interni** della rete Docker

Docker network

```
|  
|-- DNS interno  
|  
|-- server-container → 172.18.0.2  
|-- client-container → 172.18.0.3
```

Docker

■ Dockerfile

- Un Dockerfile è un documento di testo che contiene tutti i comandi che un utente potrebbe eseguire da riga di comando per assemblare un'immagine
- Contiene una sequenza di istruzioni eseguite in ordine
- Permette di automatizzare e standardizzare la creazione dell'ambiente
- Supporto la portabilità tra sistemi

■ Istruzioni principali

- **FROM** – immagine di base
- **COPY / ADD** – copia dei file
- **RUN** – esecuzione di comandi durante la build
- **EXPOSE** – porte esposte
- **CMD / ENTRYPOINT** – comando di avvio del container

Docker

- **Esempio precedente ...**
 - server-bin.c e client-bin.c

- **Dockerfile server:**

```
FROM ubuntu:22.04
```

```
# Compilatore per costruire il binario  
RUN apt update && apt install -y gcc
```

```
WORKDIR /app
```

```
# Copio il sorgente C nell'immagine  
COPY server-bin.c .
```

```
# Compilo dentro l'immagine  
RUN gcc server-bin.c -o server-bin
```

```
# Quando il container parte, esegue direttamente il server  
CMD ["/server-bin"]
```

Docker

- **Esempio precedente ...**
 - server-bin.c e client-bin.c

- **Dockerfile client:**

```
FROM ubuntu:22.04
```

```
RUN apt update && apt install -y gcc
```

```
WORKDIR /app
```

```
COPY client-bin.c .
```

```
RUN gcc client-bin.c -o client-bin
```

```
# Default: si connette a "server-cont"  
CMD ["/client-bin", "server-cont"]
```

Docker

- **Esempio precedente ...**

- server-bin.c e client-bin.c

Il file va in una **cartella dedicata**, insieme ai file da copiare nell'immagine

- **Server:**

```
es-client-server/  
└── server/  
    ├── server-bin.c  
    └── Dockerfile
```

- **Client:**

```
es-client-server/  
└── client/  
    ├── client-bin.c  
    └── Dockerfile
```

- **Immagine:**

```
cd es-client-server/server  
docker build -t c-server .
```

```
cd es-client-server/client  
docker build -t c-client .
```

Docker

- **Dockerfile**

- **Pipeline**

Dockerfile → docker build → immagine → docker run → container → processi

- **Immagine**

Il Dockerfile è il “sorgente” di un’immagine Docker

docker build -t nome .

trasforma il Dockerfile in un’immagine con quel nome

- **Container**

lanciando

docker run nome

si crea un container che esegue quella immagine

Docker

- **Dockerfile**

- **Pipeline**

Dockerfile → docker build → immagine → docker run → container → processi

- **Immagine**

```
cd es-client-server/server  
docker build -t c-server .
```

```
cd es-client-server/client  
docker build -t c-client .
```

- **Rete**

```
docker network create miarete
```

- **Container**

```
docker run --rm --name server-cont --network miarete c-server  
docker run --rm --name client-cont --network miarete c-client server-cont
```


Docker

- **Dockerfile**
- **Recuperare il PID reale del container**

```
docker inspect server-cont --format '{{.State.Pid}}'
```

- **Output**

```
2347
```

- **Esplorare in namespace reali**

```
ls -l /proc/2347/ns
```

- Ma WSL2 non li mostra ...
 - Il kernel che gestisce è **docker-desktop**